

mini project

marks prediction

importing libraries

```
In [1]: import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as s
```

reading or import dataset

```
In [2]: dataset = pd.read_csv('pd.csv')
```

```
In [5]: print(dataset)
```

	Hours	Scores
0	2.5	21.0
1	5.1	47.0
2	3.2	27.0
3	8.5	75.0
4	3.5	30.0
..	...	...
144	6.5	56.0
145	2.5	24.0
146	4.5	36.0
147	2.3	22.0
148	2.5	23.0

[149 rows x 2 columns]

```
In [7]: # Display basic information
print(dataset.shape)
print(dataset.head())
print(dataset.describe())
```

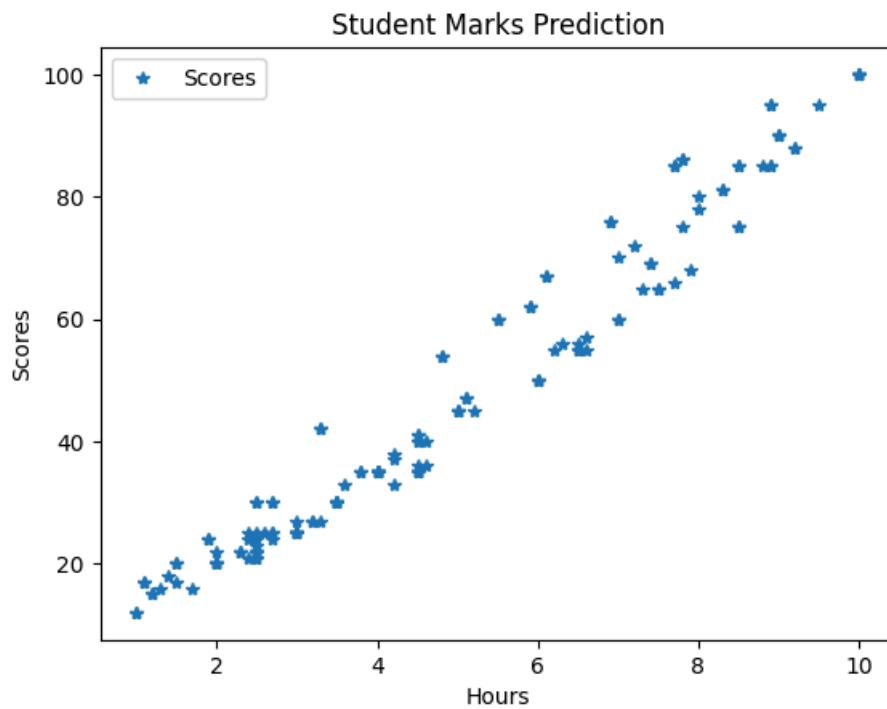
(149, 2)

	Hours	Scores
0	2.5	21.0
1	5.1	47.0
2	3.2	27.0
3	8.5	75.0
4	3.5	30.0

	Hours	Scores
count	147.000000	147.000000
mean	4.902721	46.768707
std	2.531201	25.099274
min	1.000000	12.000000
25%	2.650000	25.000000
50%	4.500000	38.000000
75%	7.000000	66.500000
max	10.000000	100.000000

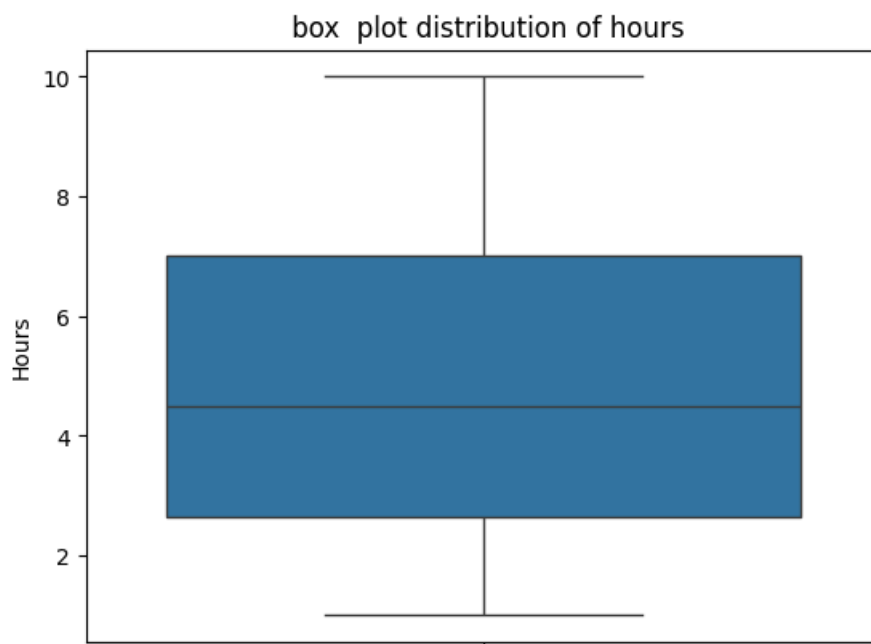
ploting

```
In [10]: # Plot the data
dataset.plot(x='Hours', y='Scores', style='*')
plt.title("Student Marks Prediction")
plt.xlabel("Hours")
plt.ylabel("Scores")
plt.show()
```

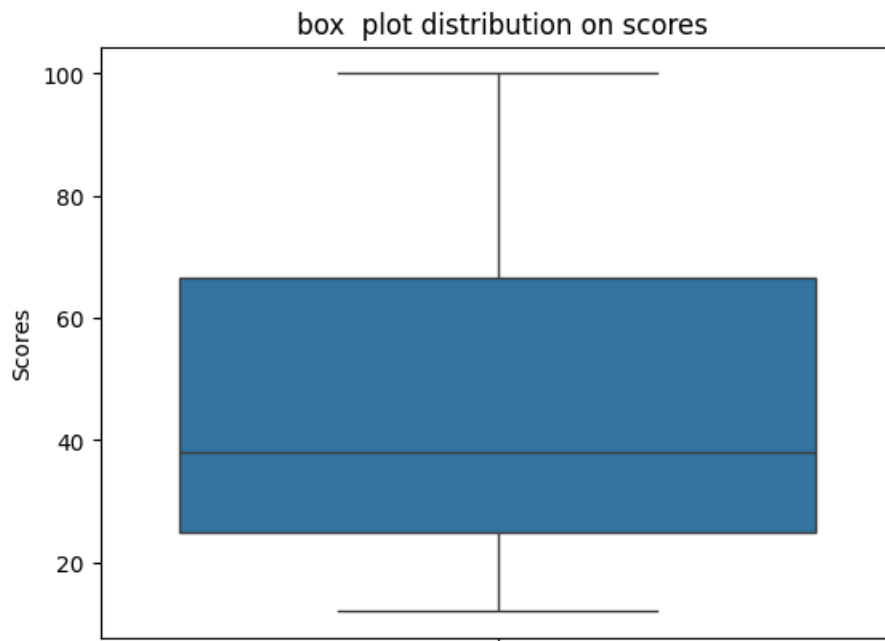


boxplot

```
In [12]: # boxplot
s.boxplot(dataset['Hours'])
plt.title(" box plot distribution of hours")
plt.show()
```



```
In [14]: # boxplot
s.boxplot(dataset['Scores'])
plt.title(" box plot distribution on scores")
plt.show()
```



missing values

```
In [16]: # missing values
a=dataset.isnull().sum()
print("missing values",a)

print("\nBefore Dropping:\n")
dataset.info()
dataset=dataset.dropna()
print('\n\nAfter Dropping:\n')
dataset.info()
```

```
missing values Hours      2
Scores      2
dtype: int64
```

Before Dropping:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 149 entries, 0 to 148
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Hours   147 non-null    float64
1    Scores  147 non-null    float64
dtypes: float64(2)
memory usage: 2.5 KB
```

After Dropping:

```
<class 'pandas.core.frame.DataFrame'>
Index: 146 entries, 0 to 148
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Hours   146 non-null    float64
1    Scores  146 non-null    float64
dtypes: float64(2)
memory usage: 3.4 KB
```

imputation

```
In [18]: print("Before Filling Null values:\n\n",dataset.isna().sum())
dataset=dataset.fillna(0)
```

```
print("\n\nAfter Filling Null Values:\n\n",dataset.isna().sum())
```

Before Filling Null values:

```
Hours      0
Scores     0
dtype: int64
```

After Filling Null Values:

```
Hours      0
Scores     0
dtype: int64
```

```
In [36]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score
# Prepare the data
X = dataset[['Hours']]
Y = dataset['Scores']

# Split the data into training and testing sets
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)

# Create and train the model
model = LinearRegression()
model.fit(X_train, Y_train)
Y_pred=model.predict(X_test)
print("mean squared error",mean_squared_error(Y_test,Y_pred))
print("r2_score",r2_score(Y_test,Y_pred))
```

```
mean squared error 25.601756451553122
r2_score 0.9593336025301838
```

finding intercept and coefficient

```
In [42]: # Display the model coefficients
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
```

```
Coefficient: [9.60324328]
Intercept: -0.2921483486029004
```

predicted and actual values

```
In [46]: # Predicting the test set results
Y_pred = regressor.predict(X_test)

# Creating a DataFrame to compare actual and predicted values
cf = pd.DataFrame({'Actual': Y_test, 'Predicted': Y_pred})

print(cf)
```

	Actual	Predicted
7	60.0	52.525690
24	86.0	74.613149
27	27.0	30.438230
119	36.0	43.882771
145	24.0	23.715960
51	25.0	28.517581
43	67.0	58.287636
22	35.0	36.200176
112	16.0	12.192068
26	47.0	48.684392
40	95.0	85.176717
78	66.0	73.652825
130	85.0	85.176717
62	85.0	81.335419
71	35.0	38.120825
91	27.0	31.398554
16	30.0	23.715960
54	50.0	57.327311
98	22.0	23.715960
8	81.0	79.414771
89	20.0	18.914338
37	41.0	42.922446
116	25.0	28.517581
84	17.0	14.112717
147	22.0	21.795311
33	81.0	79.414771
146	36.0	42.922446
90	25.0	28.517581
83	22.0	18.914338
105	12.0	9.311095

train and tested data

```
In [50]: from sklearn.model_selection import train_test_split
X = dataset.drop('Scores', axis=1)
y = dataset['Scores']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
print("Train set:", X_train.shape, y_train.shape)
print("Test set:", X_test.shape, y_test.shape)
```

Train set: (102, 1) (102,)
Test set: (44, 1) (44,)

root mean squared error and r2_score

```
In [58]: from sklearn.metrics import mean_squared_error, r2_score
import numpy as n
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)

# Create and train the model
model = LinearRegression()
model.fit(X_train, Y_train)
Y_pred=model.predict(X_test)
print(" ROOT mean squared error",n.sqrt(mean_squared_error(Y_test,Y_pred)))
print("r2_score",r2_score(Y_test,Y_pred))
```

ROOT mean squared error 5.059817827901823
r2_score 0.9593336025301838

linearregression line

```
In [60]: # Split the data into features (Study Hours) and target (Exam Scores)
X = dataset[['Hours']].values.reshape(-1,1)
y = dataset['Scores']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

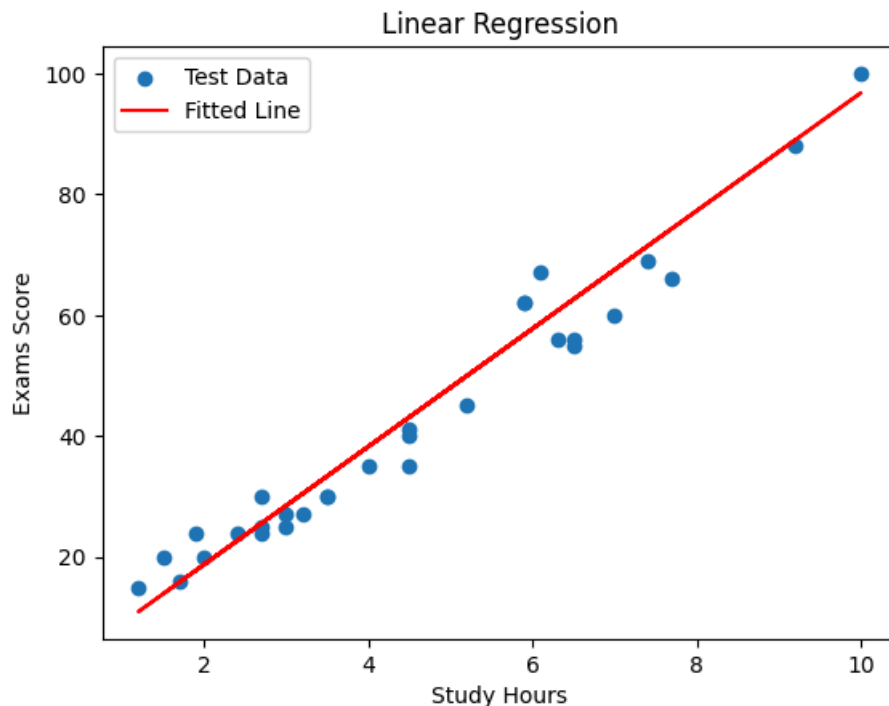
# Create a Linear Regression model
model = LinearRegression()

# Train the model on the training data
```

```

model.fit(X_train, y_train)
# Make predictions on the test data
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
# Calculate the Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)
# Plot the data points and the fitted line
plt.scatter(X_test, y_test, label='Test Data')
plt.plot(X_test, y_pred, color='red', label='Fitted Line')
plt.xlabel('Study Hours')
plt.ylabel('Exams Score')
plt.title('Linear Regression')
plt.legend()
plt.show()
print(f"Mean Squared Error: {mse:.2f}")
print(f"R-Squared Error : {r2:.2f}")

```



Mean Squared Error: 22.45
R-Squared Error : 0.95

decision tree regressor algorithm

```

In [76]: X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.3,random_state=42)
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error ,r2_score
model1=DecisionTreeRegressor()
model1.fit(X_train,Y_train)
Y_pred=model1.predict(X_test)
print("\nAccuracy of the model using Decision tree regression algorithm is ",r2_score(Y_test,Y_pred))

```

Accuracy of the model using Decision tree regression algorithm is 0.9593520554685566

RandomForestRegressor

```

In [99]: from sklearn.ensemble import RandomForestRegressor
model2 = RandomForestRegressor()
model2.fit(X_train,Y_train)
Y_pred2 = model2.predict(X_test)
print("\nAccuracy of the model using Support vector regression algorithm is ",r2_score(Y_test,Y_pred2))

```

Accuracy of the model using Support vector regression algorithm is 0.9613981688483826

support vector regerssion

```
In [78]: from sklearn.svm import SVR
model2 = SVR(kernel='rbf')
model2.fit(X_train,Y_train)
Y_pred2 = model2.predict(X_test)
print("\nAccuracy of the model using Support vector regression algorithm is ",r2_score(Y_test,Y_pred2))
```

Accuracy of the model using Support vector regression algorithm is 0.8178717287429705

SVR AND RANDOM FOREST CLASSIFIER

```
In [2]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.impute import SimpleImputer

# Load dataset
data = pd.read_csv('pd.csv')

# Independent and dependent variables
X = data[['Hours']]
y = data['Scores']

# Impute missing values in X (if any)
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# Split data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.2, random_state=42)

# Scale the data for SVM
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# SVM Model
svm_model = SVR(kernel='rbf') # Using RBF kernel
svm_model.fit(X_train_scaled, y_train)

# Decision Tree Model
dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train, y_train)

# Evaluate models (for example)
svm_predictions = svm_model.predict(X_test_scaled)
dt_predictions = dt_model.predict(X_test)

# Display predictions
print(f"SVM Predictions: {svm_predictions}")
print(f"Decision Tree Predictions: {dt_predictions}")
```

SVM Predictions: [67.08458916 54.24186515 36.94365955 65.9465831 57.62915872 66.34037189
61.64703514 43.57184063 35.05492331 63.67849389 23.47609988 39.79333365
52.51016903 24.97939656 64.30904807 61.64703514 24.50937219 53.37849042
61.58462826 66.39586975 30.54626858 24.50937219 45.38677883 23.75554781
61.64703514 57.62915872 67.1304097 30.54626858 24.97939656 23.67456101]

Decision Tree Predictions: [90. 67. 37.5 85. 55.
88. 100. 15. 35. 69.
22. 37.125 62. 26.33333333 69.
100. 23.81818182 50. 65. 86.
30. 23.81818182 47. 22.5 100.
55. 80. 30. 26.33333333 18.5]

```
In [11]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
data = pd.read_csv('pd.csv')
data_cleaned = data.dropna()
```

```

X = data_cleaned[['Hours']].values
y = data_cleaned['Scores'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
svr = SVR(kernel='rbf')
svr.fit(X_train_scaled, y_train)
y_pred_svr = svr.predict(X_test_scaled)

tree = DecisionTreeRegressor(random_state=42)
tree.fit(X_train, y_train)
y_pred_tree = tree.predict(X_test)

# Step 8: Evaluate the models
svr_mse = mean_squared_error(y_test, y_pred_svr)
svr_r2 = r2_score(y_test, y_pred_svr)

tree_mse = mean_squared_error(y_test, y_pred_tree)
tree_r2 = r2_score(y_test, y_pred_tree)

# Step 9: Print the results
print(f"Support Vector Regression (SVR) - MSE: {svr_mse:.2f}, R²: {svr_r2:.2f}")
print(f"Random Decision Tree Regressor - MSE: {tree_mse:.2f}, R²: {tree_r2:.2f}")

# Step 10: (Optional) Plot the results
import matplotlib.pyplot as plt

models = ['SVR', 'Random Tree']
mse_values = [svr_mse, tree_mse]
r2_values = [svr_r2, tree_r2]

print("train set:", X_train.shape, y_train.shape)
print("test set:", X_test.shape, y_test.shape)

# MSE plot
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.bar(models, mse_values, color=['blue', 'green'])
plt.title('Mean Squared Error', fontsize=16)
plt.xlabel('Model', fontsize=14)
plt.ylabel('MSE', fontsize=14)

# R² plot
plt.subplot(1, 2, 2)
plt.bar(models, r2_values, color=['blue', 'green'])
plt.title('R² Score', fontsize=16)
plt.xlabel('Model', fontsize=14)
plt.ylabel('R²', fontsize=14)

plt.tight_layout()
plt.show()

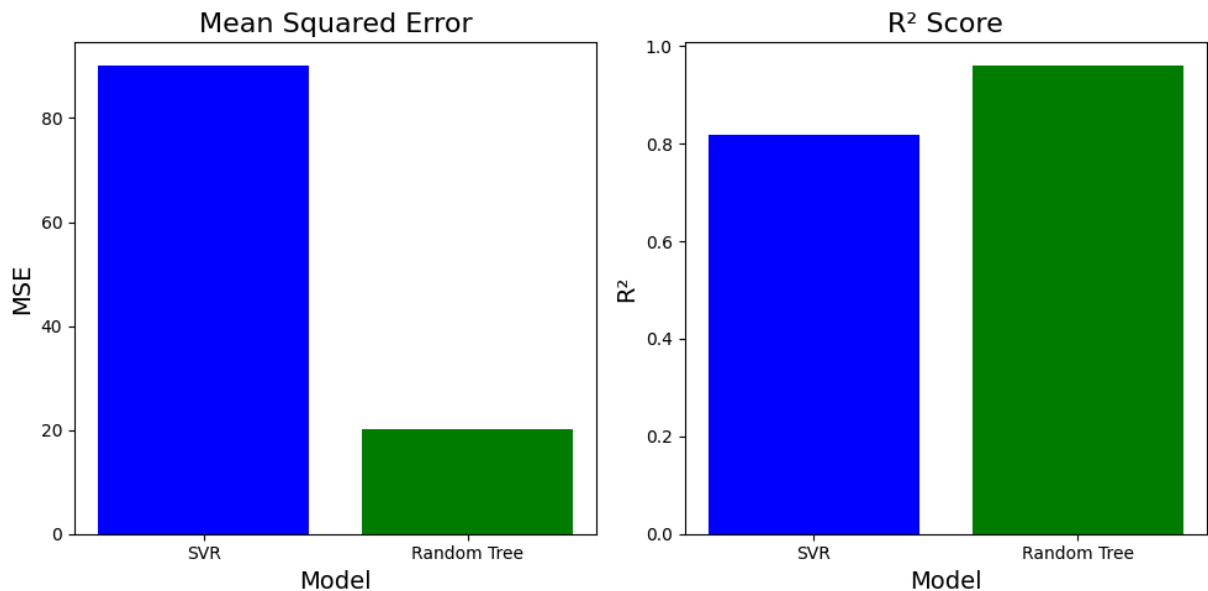
```

Support Vector Regression (SVR) - MSE: 90.05, R²: 0.82

Random Decision Tree Regressor - MSE: 20.10, R²: 0.96

train set: (102, 1) (102,)

test set: (44, 1) (44,)



LOGISTIC REGRESSION AND RANDOM FOREST CLASSIFIER

```
In [4]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# Load the data
data = pd.read_csv('Student_Marks.csv')
data['pass'] = (data['Marks'] >= 30).astype(int)
X = data[['time_study']]
y = data['pass']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print(f"\nIntercept: {model.intercept_[0]:.2f}")
print(f"Coefficient: {model.coef_[0][0]:.2f}")
plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='blue', label='Actual')
plt.scatter(X_test, y_pred, color='red', label='Predicted')
plt.xlabel('Study Time')
plt.ylabel('Pass (1) / Fail (0)')
plt.title('Study Time vs Pass/Fail: Logistic Regression')
plt.legend()
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_prob = model.predict_proba(X_range)[:, 1]
plt.plot(X_range, y_prob, color='green', label='Probability of Passing')
plt.legend()
plt.show()
```

Accuracy: 1.00

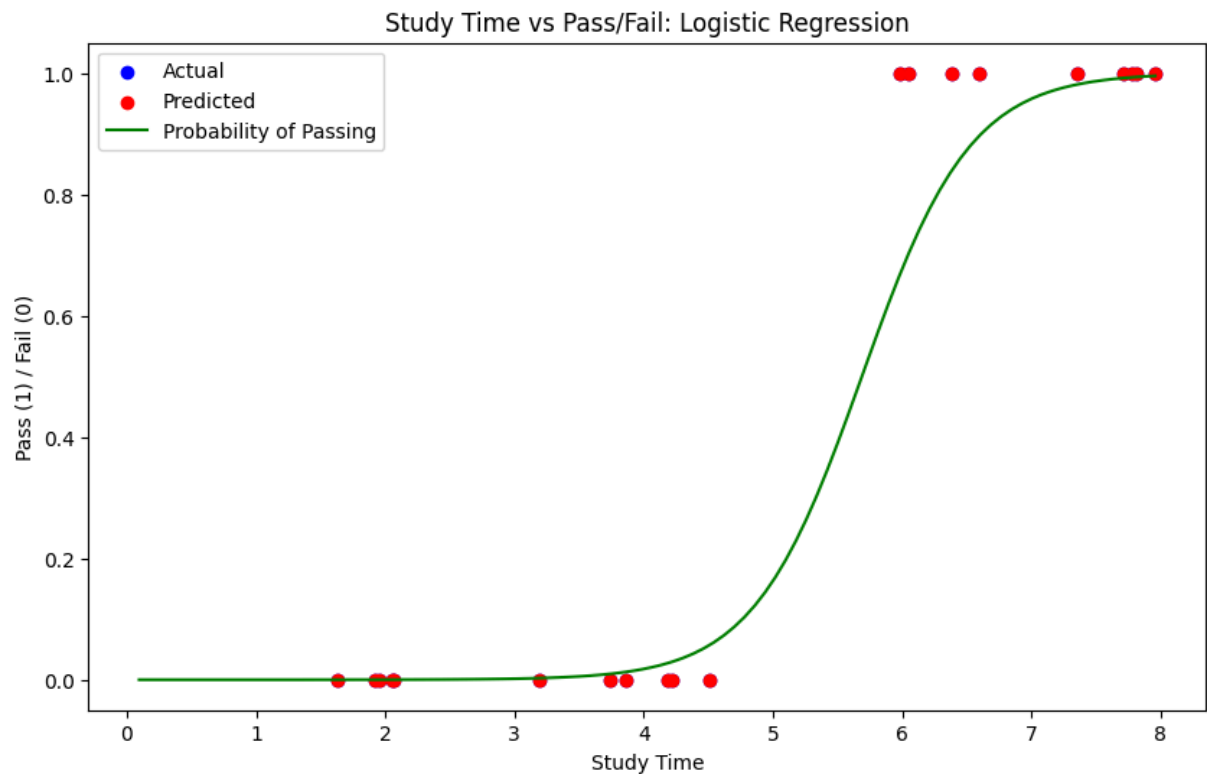
Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	1.00	1.00	9
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

Intercept: -13.53

Coefficient: 2.38

C:\Users\de11\AppData\Roaming\Python\Python312\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names
warnings.warn(



```
In [6]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# Load the data
data = pd.read_csv('Student_Marks.csv')
data['pass'] = (data['Marks'] >= 30).astype(int)
X = data[['time_study']]
y = data['pass']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='blue', label='Actual')
```

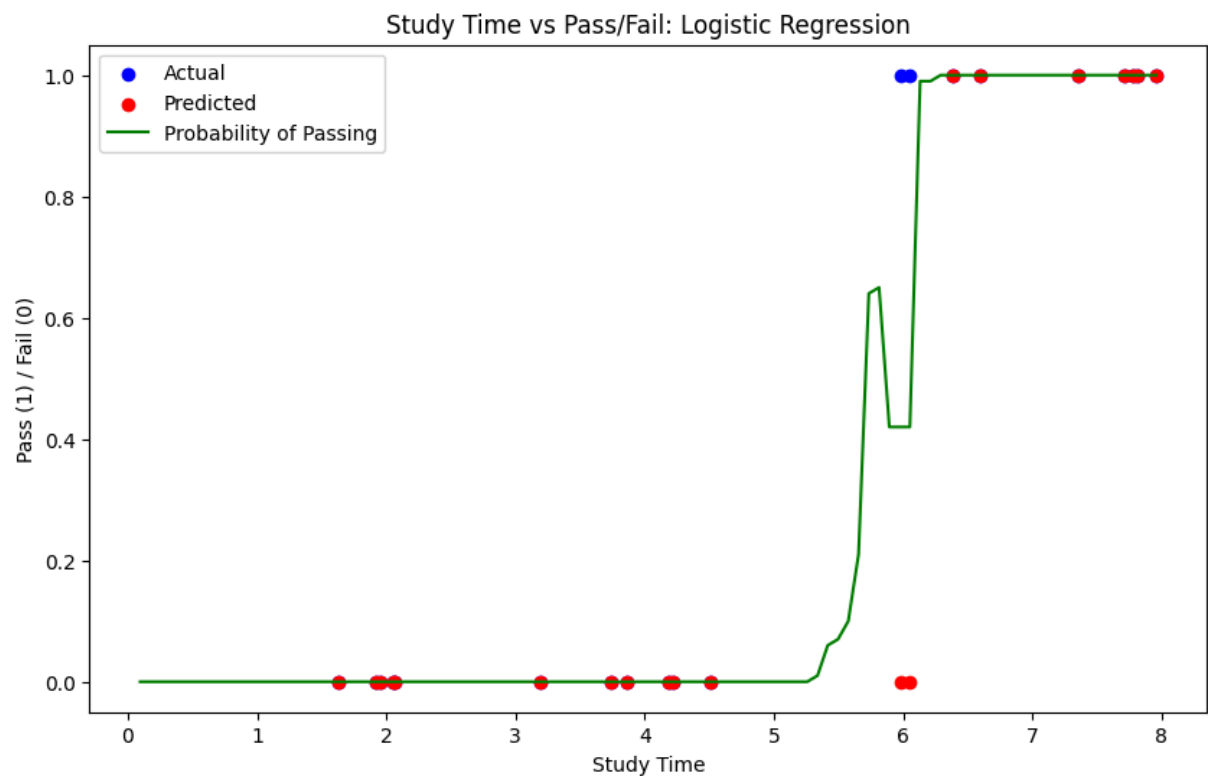
```
plt.scatter(X_test, y_pred, color='red', label='Predicted')
plt.xlabel('Study Time')
plt.ylabel('Pass (1) / Fail (0)')
plt.title('Study Time vs Pass/Fail: Logistic Regression')
plt.legend()
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_prob = model.predict_proba(X_range)[: , 1]
plt.plot(X_range, y_prob, color='green', label='Probability of Passing')
plt.legend()
plt.show()
from sklearn.metrics import mean_squared_error, r2_score
print(mean_squared_error(y_test, y_pred))
print(r2_score(y_test, y_pred))
```

Accuracy: 0.90

Classification Report:

	precision	recall	f1-score	support
0	0.85	1.00	0.92	11
1	1.00	0.78	0.88	9
accuracy			0.90	20
macro avg	0.92	0.89	0.90	20
weighted avg	0.92	0.90	0.90	20

C:\Users\de11\AppData\Roaming\Python\Python312\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names
warnings.warn(



0.1
0.595959595959596

Conclusion

apply your mini project regression to the deep learning

In [20]: *# apply your mini project regression to the deep Learning*

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

# Load the dataset
data = pd.read_csv('pd.csv')

# Display the first few rows of the dataset
print("First few rows of the dataset:")
print(data.head())

# Check for missing values
missing_values = data.isnull().sum()
print("Missing values before cleaning:\n", missing_values)

# Drop rows with missing values
data_cleaned = data.dropna()

# Verify that there are no more missing values
missing_values_cleaned = data_cleaned.isnull().sum()
print("Missing values after cleaning:\n", missing_values_cleaned)

# Split the data into training and testing sets
X = data_cleaned[['Hours']].values
y = data_cleaned['Scores'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define the model
model = Sequential()
model.add(Input(shape=(1,)))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Train the model
history = model.fit(X_train, y_train, epochs=100, batch_size=10, validation_split=0.2)

# Evaluate the model
loss = model.evaluate(X_test, y_test)
print(f"Test Loss: {loss}")

# Make predictions
y_pred = model.predict(X_test)

# Plot the results
plt.scatter(X_test, y_test, color='blue', label='Actual Scores')
plt.scatter(X_test, y_pred, color='red', label='Predicted Scores')
plt.xlabel('Hours Studied')
plt.ylabel('Score')
plt.title('Hours Studied vs Score')
plt.legend()
plt.show()

# Plot training & validation loss values
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper right')
plt.show()

```

First few rows of the dataset:

	Hours	Scores
0	2.5	21.0
1	5.1	47.0
2	3.2	27.0
3	8.5	75.0
4	3.5	30.0

Missing values before cleaning:

Hours	2
Scores	2

dtype: int64

Missing values after cleaning:

Hours	0
Scores	0

dtype: int64

Epoch 1/100

10/10  2s 34ms/step - loss: 3044.4932 - val_loss: 2618.3152

Epoch 2/100

10/10  0s 9ms/step - loss: 2644.7305 - val_loss: 2597.8933

Epoch 3/100

10/10  0s 9ms/step - loss: 2822.2046 - val_loss: 2571.6921

Epoch 4/100

10/10  0s 8ms/step - loss: 2948.4893 - val_loss: 2538.0247

Epoch 5/100

10/10  0s 9ms/step - loss: 2998.6023 - val_loss: 2493.5291

Epoch 6/100

10/10  0s 9ms/step - loss: 2778.3555 - val_loss: 2436.6106

Epoch 7/100

10/10  0s 9ms/step - loss: 2582.3398 - val_loss: 2363.3499

Epoch 8/100

10/10  0s 9ms/step - loss: 2389.2285 - val_loss: 2270.3977

Epoch 9/100

10/10  0s 21ms/step - loss: 2509.8188 - val_loss: 2155.2883

Epoch 10/100

10/10  0s 8ms/step - loss: 2262.5815 - val_loss: 2021.3428

Epoch 11/100

10/10  0s 8ms/step - loss: 2440.7549 - val_loss: 1868.0885

Epoch 12/100

10/10  0s 9ms/step - loss: 2376.6716 - val_loss: 1695.9269

Epoch 13/100

10/10  0s 8ms/step - loss: 1998.6107 - val_loss: 1519.9706

Epoch 14/100

10/10  0s 8ms/step - loss: 1676.2269 - val_loss: 1340.7585

Epoch 15/100

10/10  0s 8ms/step - loss: 1296.0944 - val_loss: 1152.5433

Epoch 16/100

10/10  0s 8ms/step - loss: 1319.1549 - val_loss: 953.0120

Epoch 17/100

10/10  0s 9ms/step - loss: 1100.8564 - val_loss: 754.9335

Epoch 18/100

10/10  0s 8ms/step - loss: 983.3992 - val_loss: 571.3548

Epoch 19/100

10/10  0s 9ms/step - loss: 763.1569 - val_loss: 411.6166

Epoch 20/100

10/10  0s 8ms/step - loss: 449.5667 - val_loss: 282.2091

Epoch 21/100

10/10  0s 8ms/step - loss: 303.2634 - val_loss: 186.7548

Epoch 22/100

10/10  0s 8ms/step - loss: 242.6036 - val_loss: 115.9030

Epoch 23/100

10/10  0s 8ms/step - loss: 148.2577 - val_loss: 72.2513

Epoch 24/100

10/10  0s 8ms/step - loss: 86.3936 - val_loss: 49.3577

Epoch 25/100

10/10  0s 9ms/step - loss: 62.7021 - val_loss: 39.9782

Epoch 26/100

10/10  0s 8ms/step - loss: 41.9738 - val_loss: 36.0593

Epoch 27/100

10/10  0s 8ms/step - loss: 37.3533 - val_loss: 33.8373

Epoch 28/100

10/10  0s 9ms/step - loss: 50.4361 - val_loss: 32.9949

Epoch 29/100

10/10  0s 10ms/step - loss: 47.1646 - val_loss: 33.0624

Epoch 30/100

10/10  0s 10ms/step - loss: 35.3074 - val_loss: 32.6388

Epoch 31/100
10/10 — 0s 20ms/step - loss: 32.5321 - val_loss: 31.8334
Epoch 32/100
10/10 — 0s 8ms/step - loss: 40.3562 - val_loss: 31.2077
Epoch 33/100
10/10 — 0s 14ms/step - loss: 34.3123 - val_loss: 30.6997
Epoch 34/100
10/10 — 0s 9ms/step - loss: 39.5972 - val_loss: 30.8660
Epoch 35/100
10/10 — 0s 9ms/step - loss: 30.0621 - val_loss: 30.4452
Epoch 36/100
10/10 — 0s 9ms/step - loss: 33.3340 - val_loss: 30.1553
Epoch 37/100
10/10 — 0s 9ms/step - loss: 36.2158 - val_loss: 29.6301
Epoch 38/100
10/10 — 0s 8ms/step - loss: 37.1254 - val_loss: 30.2000
Epoch 39/100
10/10 — 0s 9ms/step - loss: 37.5224 - val_loss: 29.5132
Epoch 40/100
10/10 — 0s 9ms/step - loss: 32.8914 - val_loss: 29.1136
Epoch 41/100
10/10 — 0s 10ms/step - loss: 32.3099 - val_loss: 28.0740
Epoch 42/100
10/10 — 0s 9ms/step - loss: 37.6460 - val_loss: 27.7938
Epoch 43/100
10/10 — 0s 10ms/step - loss: 30.9052 - val_loss: 28.1769
Epoch 44/100
10/10 — 0s 10ms/step - loss: 35.2190 - val_loss: 28.1928
Epoch 45/100
10/10 — 0s 10ms/step - loss: 33.6285 - val_loss: 27.2185
Epoch 46/100
10/10 — 0s 11ms/step - loss: 37.0172 - val_loss: 26.6875
Epoch 47/100
10/10 — 0s 14ms/step - loss: 37.9692 - val_loss: 26.0791
Epoch 48/100
10/10 — 0s 8ms/step - loss: 32.4629 - val_loss: 25.7142
Epoch 49/100
10/10 — 0s 10ms/step - loss: 28.7213 - val_loss: 25.4469
Epoch 50/100
10/10 — 0s 10ms/step - loss: 34.5268 - val_loss: 25.3847
Epoch 51/100
10/10 — 0s 10ms/step - loss: 26.8568 - val_loss: 25.3744
Epoch 52/100
10/10 — 0s 9ms/step - loss: 31.1227 - val_loss: 25.6187
Epoch 53/100
10/10 — 0s 7ms/step - loss: 36.3101 - val_loss: 24.9464
Epoch 54/100
10/10 — 0s 8ms/step - loss: 30.4444 - val_loss: 24.3105
Epoch 55/100
10/10 — 0s 9ms/step - loss: 32.3152 - val_loss: 24.9015
Epoch 56/100
10/10 — 0s 10ms/step - loss: 33.0111 - val_loss: 25.5023
Epoch 57/100
10/10 — 0s 11ms/step - loss: 29.2548 - val_loss: 24.2865
Epoch 58/100
10/10 — 0s 10ms/step - loss: 31.3384 - val_loss: 24.4602
Epoch 59/100
10/10 — 0s 10ms/step - loss: 26.7648 - val_loss: 23.3438
Epoch 60/100
10/10 — 0s 8ms/step - loss: 30.9708 - val_loss: 22.9799
Epoch 61/100
10/10 — 0s 8ms/step - loss: 24.0544 - val_loss: 22.8901
Epoch 62/100
10/10 — 0s 8ms/step - loss: 32.9856 - val_loss: 22.9069
Epoch 63/100
10/10 — 0s 9ms/step - loss: 30.9896 - val_loss: 23.0872
Epoch 64/100
10/10 — 0s 10ms/step - loss: 26.8866 - val_loss: 22.9153
Epoch 65/100
10/10 — 0s 8ms/step - loss: 31.3912 - val_loss: 24.2017
Epoch 66/100
10/10 — 0s 9ms/step - loss: 31.2421 - val_loss: 23.7482
Epoch 67/100
10/10 — 0s 8ms/step - loss: 31.4651 - val_loss: 23.4331
Epoch 68/100

```

10/10 ————— 0s 8ms/step - loss: 29.4890 - val_loss: 23.3530
Epoch 69/100
10/10 ————— 0s 16ms/step - loss: 27.6050 - val_loss: 22.3598
Epoch 70/100
10/10 ————— 0s 8ms/step - loss: 25.0380 - val_loss: 21.7144
Epoch 71/100
10/10 ————— 0s 8ms/step - loss: 31.6172 - val_loss: 20.9257
Epoch 72/100
10/10 ————— 0s 8ms/step - loss: 31.0651 - val_loss: 22.2287
Epoch 73/100
10/10 ————— 0s 14ms/step - loss: 26.9577 - val_loss: 22.0528
Epoch 74/100
10/10 ————— 0s 21ms/step - loss: 26.7491 - val_loss: 22.1828
Epoch 75/100
10/10 ————— 0s 8ms/step - loss: 24.2099 - val_loss: 21.5747
Epoch 76/100
10/10 ————— 0s 7ms/step - loss: 25.8205 - val_loss: 21.2211
Epoch 77/100
10/10 ————— 0s 15ms/step - loss: 22.9539 - val_loss: 21.0874
Epoch 78/100
10/10 ————— 0s 9ms/step - loss: 26.0222 - val_loss: 20.5323
Epoch 79/100
10/10 ————— 0s 9ms/step - loss: 25.9803 - val_loss: 20.6752
Epoch 80/100
10/10 ————— 0s 8ms/step - loss: 32.6101 - val_loss: 22.1048
Epoch 81/100
10/10 ————— 0s 7ms/step - loss: 26.8934 - val_loss: 22.3190
Epoch 82/100
10/10 ————— 0s 18ms/step - loss: 24.4759 - val_loss: 22.4240
Epoch 83/100
10/10 ————— 0s 7ms/step - loss: 28.4439 - val_loss: 21.7266
Epoch 84/100
10/10 ————— 0s 7ms/step - loss: 25.6677 - val_loss: 20.8451
Epoch 85/100
10/10 ————— 0s 7ms/step - loss: 26.5716 - val_loss: 20.1702
Epoch 86/100
10/10 ————— 0s 7ms/step - loss: 28.1618 - val_loss: 20.0527
Epoch 87/100
10/10 ————— 0s 7ms/step - loss: 27.8400 - val_loss: 20.3307
Epoch 88/100
10/10 ————— 0s 7ms/step - loss: 29.1337 - val_loss: 20.6798
Epoch 89/100
10/10 ————— 0s 8ms/step - loss: 31.5133 - val_loss: 21.2635
Epoch 90/100
10/10 ————— 0s 17ms/step - loss: 24.8857 - val_loss: 20.5917
Epoch 91/100
10/10 ————— 0s 7ms/step - loss: 24.8469 - val_loss: 21.1764
Epoch 92/100
10/10 ————— 0s 8ms/step - loss: 27.7018 - val_loss: 20.6349
Epoch 93/100
10/10 ————— 0s 7ms/step - loss: 28.9770 - val_loss: 20.7366
Epoch 94/100
10/10 ————— 0s 9ms/step - loss: 27.9701 - val_loss: 21.7452
Epoch 95/100
10/10 ————— 0s 9ms/step - loss: 28.7701 - val_loss: 21.1224
Epoch 96/100
10/10 ————— 0s 10ms/step - loss: 24.7011 - val_loss: 20.9262
Epoch 97/100
10/10 ————— 0s 20ms/step - loss: 30.2346 - val_loss: 21.4454
Epoch 98/100
10/10 ————— 0s 15ms/step - loss: 29.4579 - val_loss: 20.6490
Epoch 99/100
10/10 ————— 0s 12ms/step - loss: 27.0953 - val_loss: 19.7823
Epoch 100/100
10/10 ————— 0s 7ms/step - loss: 26.5394 - val_loss: 19.3059
1/1 ————— 0s 51ms/step - loss: 18.5868

```

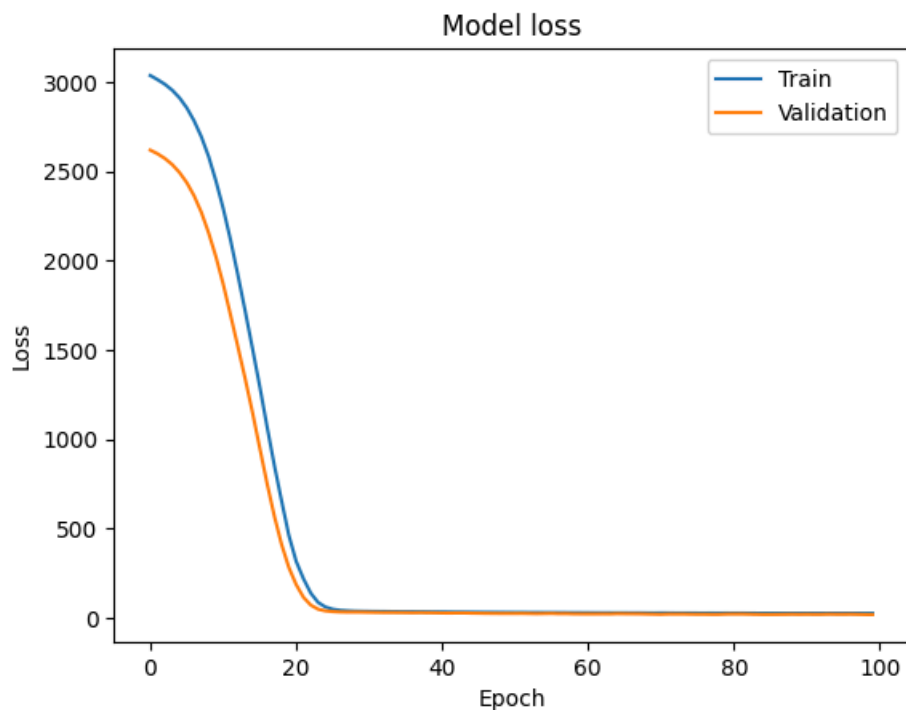
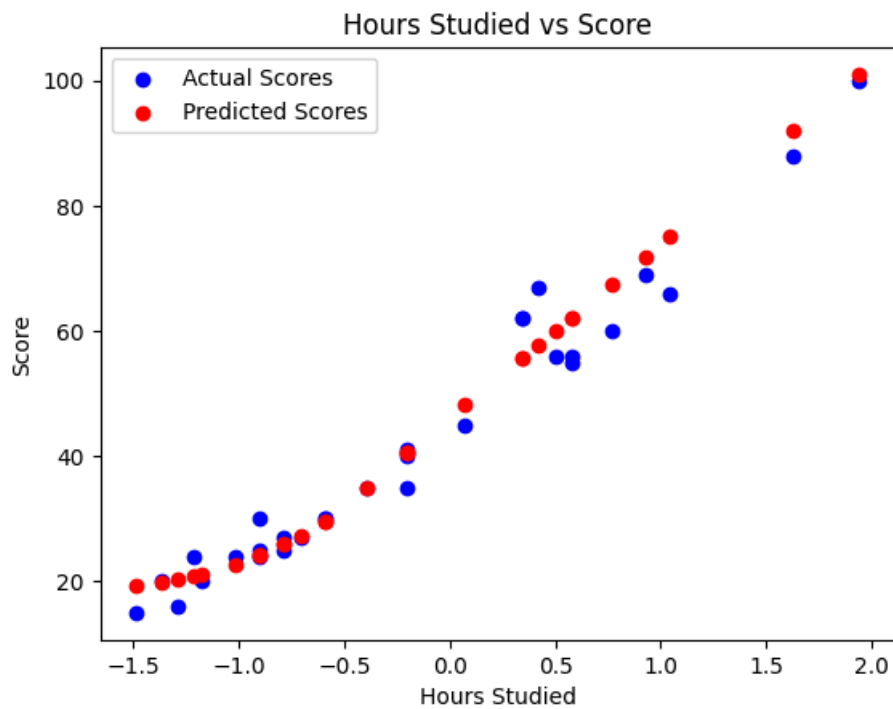
Test Loss: 18.58681297302246

WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_function.<locals>.>.one_step_on_data_distributed at 0x0000024C67003740> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

```

1/1 ————— 0s 98ms/step

```



I have successfully built a simple linear regression model to predict scores based on the number of hours

The model predicted a score of approximately 92.39 for a student who studies for 9.25 hours/day.

The model's performance is good. With low error metrics (

(Mean Absolute Error, Mean Squared Error) and a high R-squared score, it accurately predicts student scores based on study hours.

In []: